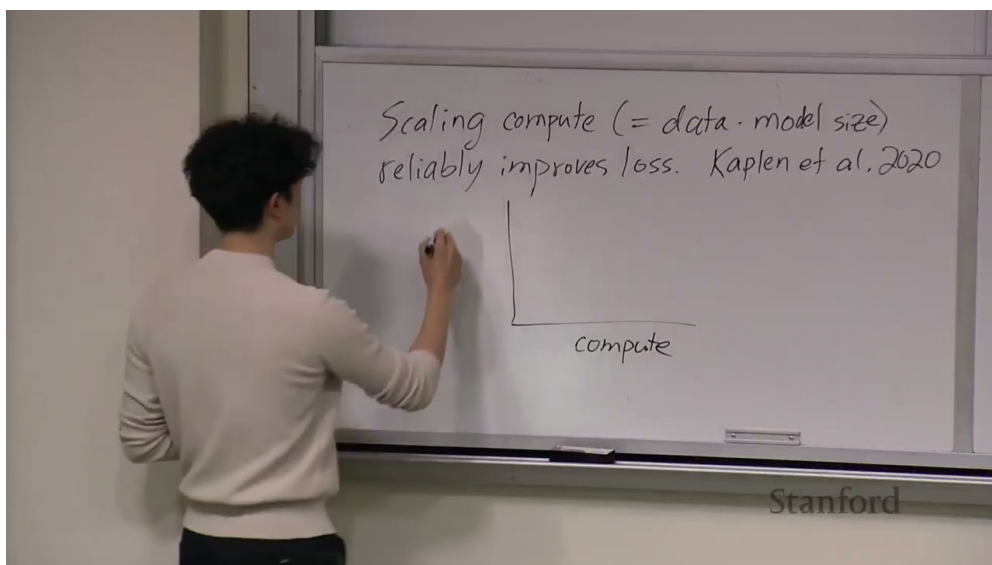


课程笔记

[CS25] Intuitions on LMs + History of Architectures — Jason Wei + Hyung Won Chung, OpenAI

基于 Jason Wei 与 Hyung Won Chung 在 Stanford CS25 Transformers United 公开讲座
整理

2024 年 04 月 26 日



视频作者/频道: Stanford CS25: Transformers United

发布日期: 2024 年 04 月 26 日

视频时长: 约 1 小时

视频链接: <https://www.youtube.com/watch?v=5XkoZDxBSx0>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：讲座定位与结构化复现	3
1.1 课程概览	3
1.2 素材与结构策略	3
1.3 讲座时间线	3
1.4 本章小结	4
2 数据直觉与 emergent 能力	4
2.1 理解数据即理解模型	4
2.2 Manual data review 与 emergent cues	4
2.3 Emergent phenomena 的 evidence stack	4
2.4 多模态 prompt shift	5
2.5 数据治理与规范化	5
2.6 数据可观测性与工具	5
2.7 本章小结	6
3 提示工程与 reasoning orchestration	6
3.1 Instruction tuning 是格式化而非新能力	6
3.2 Chain-of-Thought 与 Self-Consistency	6
3.3 Prompt 仪表盘	7
3.4 Prompt Stress Testing	7
3.5 本章小结	8
4 架构演化与归纳偏置	8
4.1 The Bitter Lesson 与计算优先	8
4.2 从 encoder-decoder 到 decoder-only	8
4.3 训练目标的变化	8
4.4 Scale Law 与优化器选择	8
4.5 本章小结	9
5 评估、监控与治理	9
5.1 多维度评估矩阵	9
5.2 评估数据管道与报警	9
5.3 Hallucination taxonomy	10
5.4 本章小结	10
6 部署、复现与团队实践	10
6.1 部署准备 checklist	10
6.2 SLI/SLO 与成本控制	11
6.3 Incident Response 与 grade-book	11
6.4 Tooling stack 与 observability	11
6.5 复现 pipeline 与知识管理	11
6.6 智能体典型案例	12
6.7 团队角色与知识共享	12

6.8 本章小结	12
7 案例驱动演练	12
7.1 Emergent detection drill	12
7.2 Agent safety drill	13
7.3 本章小结	13
8 Q&A 与实践提醒	13
8.1 现场问答精华	13
8.2 课堂 takeaways	14
8.3 本章小结	14
9 术语与指标速查	14
9.1 关键术语	14
9.2 常用指标速查	14
9.3 本章小结	15
10 行动与落地检查清单	15
10.1 短期行动	15
10.2 长期研究议程	15
10.3 本章小结	16
11 Emergent 能力记分板	16
11.1 任务追踪面板	16
11.2 指标联动与 drift analysis	16
11.3 本章小结	16
12 Observation daybook 记录	17
12.1 关键事件记录模板	17
12.2 Lessons learned 速查	17
12.3 本章小结	17
13 团队承诺与治理约定	17
13.1 治理角色 RACI	17
13.2 本章小结	18
14 总结与延伸	18
14.1 拓展阅读	19

1 引言：讲座定位与结构化复现

1.1 课程概览

Lecture 27 由 Jason Wei 与 Hyung Won Chung 联合讲解：前者围绕“为什么语言模型表现如此出色”分享了大量来自预训练数据与 prompt 的直觉，后者围绕架构演化、归纳偏置与“苦涩的教训”提供了数学与工程视角。两位讲者在 Q&A 环节共同强调：LLM 的力量源于简单方法 + scale + 数据治理。

1.2 素材与结构策略

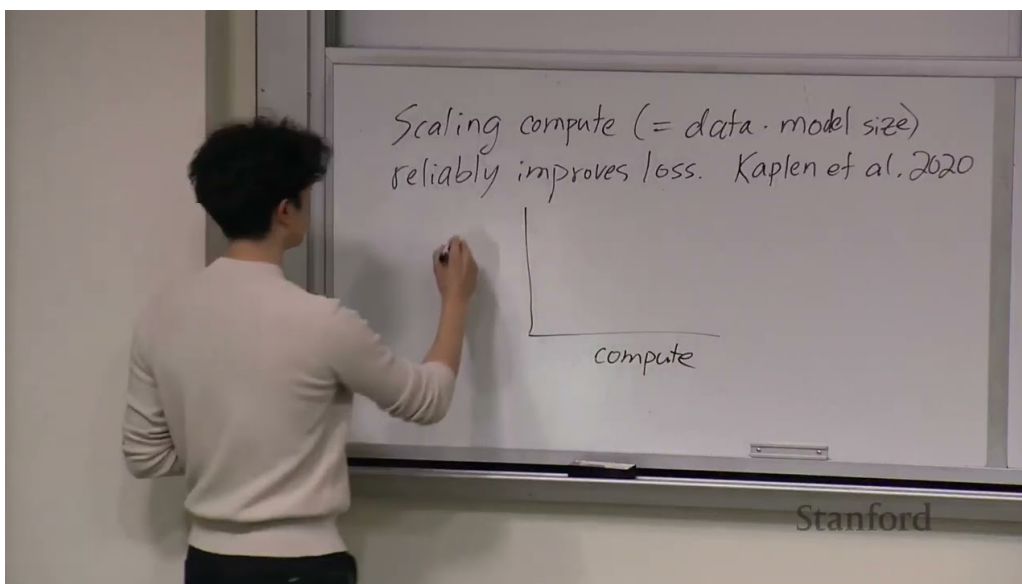


图 1: 课程封面 / slide (00:00:05–00:00:25)。

复现素材原则

优先引用讲者提供的 slides 与关键截图；如无可用图片，则通过表格、流程图与 Box 结构重现讲述顺序，并明确列出时间段与关键词，方便读者在原视频中对照。

1.3 讲座时间线

时间段	内容概览
00:00–00:09	对 Jason 的介绍、LLM 的 emergent ideas、数据直觉
00:09–00:24	Instruction tuning、Chain-of-Thought 与 prompt metrics
00:24–00:38	Hyung Won 讲述 Bitter Lesson、架构演化与归纳偏置争论
00:38–00:55	自回归/decoder-only 的扩展、训练目标对比
00:55–01:02	Q&A、实践建议、治理与复现 checklist

表 1: Lecture 27 的时间线与章节分布。

⁰封面来自 Stanford CS25 公开 slides，后续如有其他 slide 图像请放在本目录 ‘slides-images/’ 并保持命名一致。

时间线的使用方式

把时间段和主题写入笔记有助于复盘：每次看视频只需跳到本表对应区间即可；若需要深入分析某一个主题，可结合 timecode+table 快速定位 slide/quote。

1.4 本章小结

本节说明本讲的双轨策略：Jason 负责 data/prompt/performance 的直觉，Hyung Won 则把架构与归纳偏置放到长期 scaling 的历史脉络中；整个笔记按教学逻辑重新拆解，方便后续复现与治理。

2 数据直觉与 emergent 能力

2.1 理解数据即理解模型

Jason Wei 强调：LLM 的能力不是来自某个新架构，而是数据里的隐式“任务”。他亲手翻阅训练语料，发现其中含有翻译、数学解题、代码片段、对话、百科等多种模式；语言建模的目标驱使模型为每种模式学会不同的技能。

语言建模的多任务隐藏层

LLM 的下一个 token 预测需要整合语法、语义、常识推理和 task formatting，因此它从数据中隐式吸收了“翻译、推理、编程、对话”等任务。如果数据缺失，模型在对应能力上会显著退化。

2.2 Manual data review 与 emergent cues

Jason 详细介绍他在训练前走一遍数据的习惯：把训练语料分成 chunk、人工审阅特殊 tokens、提取 multi-turn 交互。通过这样的工作，他归纳出几点 emergent cue，鼓励团队把相同的流程复制到自己的项目。

人工数据审查的五个步骤

1. 抽样多语言文本、文档、对话
2. 统计 token frequency 与 rare token 列表
3. 识别长链条的推理与步骤式文本
4. 标注 prompt-like 标题（“Step 1/2”）与答案分隔符
5. 用轻量脚本生成 attention map（或 heuristics）以发现分布 shift

2.3 Emergent phenomena 的证据堆栈

Lecture 27 重复出现这个核心句：“Emergence is real when multiple evidence lines all shift at once.” Jason 建议同时观察（1）loss/perplexity 曲线、（2）prompt trace（是否需要 CoT）、（3）

attention entropy 以及 (4) 抽样输出的 stability; 只有多个维度协同跳跃时, 才能断言一个能力是真正 emergent。

误判 emergent 的三个陷阱

- 只看 accuracy/精确匹配; 可能是 metrics 过于敏感
- 单次 trial 的涨幅, 很可能是 noise
- 没有控制 prompt/system message 的时候, 无法复现

2.4 多模态 prompt shift

Hyung Won 补充: 当把图像、音频转成 token 时, 同样可以看到涌现; 只是引入新的 attention bias (如 per-modality token embeddings) 后, 需要稳定 prompt shift (“先识图再问”)。

多模态 emergent 的提示实践

- 用不同 token prefix (“[image]”, “[audio]”) 将模态区分开
- 设定约束 (“先描述再回答”) 减少 hallucination
- 观察 attention heatmap 是否对齐到 vision token, 而不是仅靠 text

2.5 数据治理与规范化

Jason 强调 emergent 研究必须先把 data pipeline 定义清楚: 每个 dataset 都要记录采集 source、split 逻辑、dedup 策略与 sensitive content filter, 以便后续复现或提供 compliance report。

字段	描述
Source	抽样 website / corpus / internal doc
Split	train/validate/test 的 token 分割方式
Dedup	ngram overlap 或 semantic hash 过滤 阈值
Safety	黑名单、PII scrub 策略

表 2: 数据治理时需记录的关键属性。

数据治理的价值

在 emergent 现象触发后, 团队需要回溯 “是哪批数据” 使模型跳变; 如果没有详细治理记录, 就无法判定是否引入有问题的样本。

2.6 数据可观测性与工具

为了让 emergent 现象可追溯, Jason 建议把数据采样、attention map 和 prompt trace 交给统一平台。下表展示了一个轻量级数据观测 pipeline:

组件	描述
Sampler	定期从数据 lake 中抽样多模态 chunk (text/image/code)
Instrumenter	记录 attention map、CoT trace、token distribution
Dashboard	显示 loss/perplexity + drift metric, 支持 drill-down

表 3: 简单的数据可观测 pipeline。

数据仪表盘的价值

把 instrumentation + dashboard 链接到 prompt log, 可以在 emergent drift 发生时立即 trace 到具体数据 batch, 而不是仅依赖后验分析。

2.7 本章小结

本节将 emergent ability 的经验归结为“写清数据 + prompt + attention + metrics”四要素；如果只追求参数量而不排查上述维度，就难以复现 Jason 所描述的现象。

3 提示工程与 reasoning orchestration

3.1 Instruction tuning 是格式化而非新能力

Jason 把 Instruction Tuning 描述为“让模型把已有知识用新的纸张打印出来”。以 FLAN/PaLM 为例，单纯在多任务指令上 fine-tune，模型在零样本任务上的表现即可立刻提升；说明模型底层已经具备能力，只需要 prompt 来调度。

Instruction tuning 的常见误区

- 误以为 tuning 训练了新能力，其实只是在改变 sampling distribution
- 过度依赖 few-shot 示例，导致 context window 被 prompt 占满
- 忽略 prompt drift，改了 prompt 但不记录版本，就看不到 performance 变化

3.2 Chain-of-Thought 与 Self-Consistency

CoT 的本质是“prompt 里先给出思路，再让模型重演”。Self-Consistency 进一步通过采样多个 CoT 路径并投票来减少偶发错误。

方法	核心机制	典型收益
Chain-of-Thought	给出 step-by-step	解题准确率 +15%
Self-Consistency	多路径 majority vote	抑制 single path failure
Least-to-Most	分层 prompts	适合长链任务

表 4: 讲者提及的三种推理提示策略。

提示工程的治理建议

把每次 prompt/CoT pattern 写进 prompt log (格式: Prompt ID、Role description、Examples、Sampling parameters、Metric diffs), 并与 evaluation dashboard 里的 drift metric 关联; 一旦 accuracy 指标下降, 可以凭 log 快速回滚。

3.3 Prompt 仪表盘

Jason 进一步建议在 prompt log 上挂上 `self_consistency_score`、`hallucination_rate`、`token_budget` 和 `sampling_temperature` 等字段, 方便分析 prompt drift。

字段	说明
Prompt ID	Git tag/hash, 便于复现
Role + Examples	CoT 模板与 role-play
Sampling params	temperature / top_p / candidate count
Metric diffs	前后 accuracy/self-consistency 对比

表 5: Prompt 仪表盘中需要记录的关键字段。

3.4 Prompt Stress Testing

讲者强调: 每次 prompt 改动都应经历 “stress testing” —— 选取 hardest-case prompts、insert adversarial tokens、vary temperature/backoff 以观察 performance drop。对于 prompt drift, 应提前设定 rollback plan。

Prompt stress testing checklist

- Run CoT prompts under different seeds/temperature; check self-consistency variance
- Inject adversarial tokens or hallucination triggers; confirm no uncontrolled behavior
- Monitor token budget + latency to prevent prompt size blow-up

Prompt regression guardrail

把 stress testing 的结果记录在 prompt log, 对比前后 accuracy/self-consistency; 若发现 degradation > threshold, 即刻回滚到先前的 prompt 版本。

3.5 本章小结

Prompt 不是一次性输入，而是人与模型之间的 orchestration：只有把 instruction tuning、CoT、dashboard 与 metric drift 结合，才能把 emergent ability 稳固下来。

4 架构演化与归纳偏置

4.1 The Bitter Lesson 与计算优先

Hyung Won 引用了 Sutton 的 “The Bitter Lesson”，说明 “用计算而不是手工归纳” 是达成长期 scaling 的唯一路径。工程师需要问自己：“我正在往模型里注入多少归纳偏置？是否可以通过 scale + data + compute 达成同样效果？”

Sutton 的三条启示

- 人类知识在有限 compute 下有效，但不具备 scaling
- 更泛化的模型往往是 compute-intensive 而非 architecture-heavy
- 归纳偏置想增加但也要有退出策略 (revert to simpler architecture when compute grows)

4.2 从 encoder-decoder 到 decoder-only

Hyung Won 沿着历史讲述了 Encoder-Decoder (2017)、BERT (2018)、GPT (2018+) 的演化。Decoder-only 架构的核心好处在于 KV cache，可让多轮对话快速复用之前的 key/value，从而拉低推理 FLOPs。

双向注意力的伸缩问题

BERT 等编码器架构需要重新计算全部 attention，因此在长 context 中开销爆炸；decoder-only 通过因果 mask + KV cache，令训练和推理都具备线性扩展性。

4.3 训练目标的变化

目标	架构	主要假设
Masked LM	Encoder	需要双向上下文
Prefix LM	Encoder-Decoder	encoded prefix + causal decode
Autoregressive LM	Decoder-only	只看过去，易 scale

表 6: 几种主流训练目标与对应架构的区别。

4.4 Scale Law 与优化器选择

Hyung Won 指出，scale law 表示 loss 与 compute 之间呈幂律关系，因此进入 scaling regime 时必须重新调优 optimizer + lr schedule。典型流程是：先用 AdamW + cosine decay 进行 warmup，再根据 attention scale 观察是否需要进一步减小 lr 或增加 batch。

Scale law 经验值

- $\text{loss} \approx \text{constant} \times \text{compute}^{-0.05}$, $\text{compute} = \text{params} \times \text{tokens} \times 6$
- batch 太小无法进入 regime, 需 gradient accumulation
- 监控 attention scale, 有助于判断是否需要 LR warmup/stretch

优化器与 warmup 策略

- AdamW + cosine decay 是 transformer scaling 的常用组合
- warmup 阶段要缓慢把 lr 提升到 target, 避免 early loss 爆炸
- 遵循 linear scaling rule: $\text{batch} \times \text{lr} \approx \text{常数}$, 便于增加 batch 时同步调参

归纳偏置的判断准则

如果一种结构需要“人为指定 how to attend”, 可能正在引入归纳偏置; 若它只是提供更少的假设 (如 decoder-only), 就更容易 scale。

4.5 本章小结

架构演化是一条“减少人为假设 + 增加 compute”的路径: 讲者用实证与经验告诉我们, 在面向 scaling 的时候, 最可靠的策略是先移除结构, 再投放计算。

5 评估、监控与治理

5.1 多维度评估矩阵

为了避免 emergent drift, Jason 设计了一个评估矩阵: 准确率/任务表现、prompt drift、hallucination rate、token budget、CoT fidelity 等需要同时追踪。

维度	指标举例	工具
准确率	BLEU/F1/custom	Eval harness
鲁棒性	Adversarial prompt	adversarial suite
Hallucination	Fabrication rate	human annotator + knowledge graph check
Prompt drift	Self-consistency drop	versioned dashboard

表 7: 智能体/LLM 的评估矩阵。

5.2 评估数据管道与报警

讲者建议把评估 pipeline 分成 instrumentation、aggregation、alerting 三段: 先采集 prompt+tool 调用 +attention snapshot, 再把 metrics 送入 dashboard, 最后设定阈值 triggers 及时通知。

评估 data pipeline 的三步

- Instrumentation: 每次 infer 记录 prompt、tool、attention map
- Aggregation: 定期刷新 metrics 看是否 drift, 并与 compute budget 对齐
- Alerting: Accuracy or hallucination 超阈值时发起 rollback

5.3 Hallucination taxonomy

为了治理 hallucination, Jason 建议把 hallucination 分为 three classes: fact-unsupported、failure to recall context、and prompt-induced hallucination。每种类型都需要不同的反馈 loop, 例如 fact-unsupported 需要 knowledge graph check, prompt-induced 需要 prompt stress testing。

Hallucination 分类与 FF

- fact-unsupported: vs knowledge graph + web verification
- context loss: 增加 context window & caching
- prompt-induced: 锁定 role + CoT template

5.4 本章小结

评估不是单一指标, 而是可审计、可回溯的闭环; 数据 pipeline + dashboard + alerting 才能在 emergent world 里保持可控。

6 部署、复现与团队实践

6.1 部署准备 checklist

Jason 和 Hyung Won 共同强调: 部署前必须校验 prompt stability、token budget、fallback 策略与监控 hook。

项目	核心检查	状态
Prompt 稳定性	不同输入的 Self-consistency	已就绪
Token 预算	每次请求 FLOPs 对齐 compute cap	已就绪
Fallback	API / 缓存降级方案	待完善
Monitoring hook	latency/loss/attention	已就绪

表 8: 部署前必须校验的关键指标。

6.2 SLI/SLO 与成本控制

SLI/SLO 参考

- Latency SLO: 99
- Accuracy SLI: Self-Consistency+CoT 精度达标
- Token budget: 每月消耗不超预留

6.3 Incident Response 与 grade-book

Hyung Won 提到: 治理 emergent drift 需要 “incident grade-book”, 记录 Alarm、Root cause、Resolution。每次 incident 都要归档 prompt log、evaluation snapshot 与 fix script, 避免重复 error。

Incident grade-book 示例字段

- Alarm trigger: self-consistency drop / hallucination ratio spike
- Root cause: prompt drift / data shift / compute overload
- Resolution: rollback prompt / retrain / add guardrail

6.4 Tooling stack 与 observability

部署 pipeline 应覆盖 prompt log、attention heatmap、latency/failure metrics, 再加上 tool call trace (doc search、python 执行)。

层级	工具/实践
Prompt	Prompt log + versioned dashboard
Inference	Monitoring hook + latency/failure metrics
Tooling	API call trace + sandbox constraints
Observability	Attention/Config snapshot + drift alert

表 9: 部署时的 tooling stack。

6.5 复现 pipeline 与知识管理

复现实践的沉淀机制

- 把 prompt log + config + sample output 写入 ‘notes/’ 目录
- 用 ‘git tag’ 记录 stable prompt/config, 并排期 review
- 维护 ‘slides-images/’ 与 ‘frames/’, 让团队新人快速上手

6.6 智能体典型案例

讲者以“自动研究助理”为例：输入需求后拆成子任务，依次调用 doc search、python executor、web fetch，再由 human review + self-consistency 检查结果，完成报告。每个工具调用前插入 reasoning trace 以防 hallucination。

阶段	人机协作
需求理解	Prompt 拆分任务、assign role
工具调用	Doc search + code executor + web search
结果验证	Human review + self-consistency votes

表 10: 自动研究助理的工作流。

6.7 团队角色与知识共享

部署和复现需要跨角色协作：Jason 指出一个典型团队包括 data engineer、model engineer、prompt engineer 与 evaluator。每次 run 都必须写入 ‘prompt log’, ‘attention snapshot’, ‘grid search summary’, 以便后续 replicate。

团队协作的 artifact

- data engineer: dedup log、tokenizer config
- model engineer: training log、attention heatmap、scale law plots
- prompt engineer: prompt versions、CoT templates、stress testing reports
- evaluator: hallucination annotation + drift alert summary

6.8 本章小结

部署与复现不是补充，而是主线：从 SLI/SLO 到 prompt log 再到自动化案例，每一步都要写实并可追踪。

7 案例驱动演练

7.1 Emergent detection drill

设计 emergent drill 时要把 “baseline metric + prompt guardrail” 视为 control group，再定义 “emergent metric”（如 multi-step accuracy × CoT fidelity）作为 treatment。Jason 建议每周开展一次 drill，把 emergent jump、hallucination sample 以及 prompt drift log 一起分析。

组件	操作
Control metrics	loss/perplexity + token budget
Treatment metric	multi-step accuracy × self-consistency
Drill cadence	每周一次，包含 prompt log + attention snapshot

表 11: Emergent detection drill 的结构化流程。

Drill 的目的

通过 structured drill 快速判断 emergence 是否真实：若 multi-step accuracy 与 CoT fidelity 同步提升且 hallucination rate 不升高，则可认为 emergent 已经稳定。

7.2 Agent safety drill

Hyung Won 建议把 agent safety 训练成 “prompt + tool + fallbacks” 的组合：每次施加 adversarial instruction (例如 ‘delete all files’), 观察 guardrail 是否及时触发；若没有，必须更新 reasoning trace 的 ‘stop tokens’。

安全演练清单

- 针对 high-risk commands 触发 multi-factor confirmation
- 检查 tool call 是否在 sandbox 执行，防止 data leak
- 记录 reasoning trace 和 human review log 以备审计

7.3 本章小结

案例演练让 emergent ability 与 agent safety 不再是 abstract theory，而是每周 checklist；通过 structured drill，团队可以把 observed drift 及时转化为 governance action。

8 Q&A 与实践提醒

8.1 现场问答精华

在 Q&A 环节，Jason 重申 “emergent ability 需要 multi-metric evidence”，Hyung Won 回答 “reduce structure, ramp compute”。他们还强调 prompt log 要包含 ‘role’、‘CoT template’、‘version hash’、‘mixture of few-shot + zero-shot’ 化的配比。

实践提醒

- 记录 prompt version 及 sampling 时的 seed/temperature
- 每次 major drift 都要有 artifact (prompt + dataset + attention map)
- 让 evaluator 直接在 dashboard 点出 drift case 并发起 alert

8.2 课堂 takeaways

讲者最后建议：把 emergent/agent research 做成 “observation → instrumentation → governance” 的闭环，并让整支 team 每周 review 一次 incident grade-book。

Takeaway checklist

- Observe emergent jump across metrics before declaring success
- Instrument prompt/data/code artifacts with per-run metadata
- Govern via prompt log, incident grade-book, evaluation dashboard

8.3 本章小结

Q&A 的重点在于把上文提到的 artifacts 串成闭环：观察 → 记录 → 治理，让 emergent/agent 研究能在可控范围内不断迭代。

9 术语与指标速查

9.1 关键术语

术语	释义
CoT	Chain-of-thought，向模型展示完整推理步骤
Self-consistency	多路径采样后取众数，减少偶发错误
Governance loop	Prompt log + dashboard + incident grade-book
KV cache	Key/value 缓存，用于 decoder-only 多轮对话加速

表 12: Lecture 27 重复出现的几个关键术语。

9.2 常用指标速查

指标	用途
Self-consistency score	评估 prompt/draft 是否一致
Hallucination rate	记录 fabrication 事件占比
Token budget	每次请求的 token/compute 限额
Attention entropy	判断模型是否跳到不同模式

表 13: 治理与调试时常用的指标。

9.3 本章小结

把术语与指标写下来，便于新成员快速理解讲座用语，也可以在 incident review 中快速定位 key term。

10 行动与落地检查清单

10.1 短期行动

Jason 建议把 emergent/agent 研究拆成短期行动：4 周内完成 data review + prompt stress testing；8 周内搭建 evaluation dashboard；12 周内形成 incident grade-book。

时间	任务
Week 1-4	Finish data review + set prompt stress test suite
Week 4-8	Launch evaluation dashboard + drift alerts
Week 8-12	Establish incident grade-book + governance review

表 14: 短期行动计划。

短期行动的关键提醒

- 每次 prompt change 都要关联 version + drift metric
- Drift alert 触发后立即记录 ‘incident grade-book’
- 让 evaluator 跟 data engineer 同框 review 结果，增加 cross-check

10.2 长期研究议程

Hyung Won 鼓励团队关注 “reduce inductive bias”、“scale law mapping”、“attention interpretability” 三件长期项目。这些研究最好以 quarterly thesis 形式展开，每个项目都放在 README/notes 中。

长期议程示例

- Project Alpha: 实验不同注意力结构，验证是否能保持 performance 且 reduce bias
- Project Beta: 把 scale law 曲线映射到 optimizer setting，为 large-scale training 预先设定 lr schedule
- Project Gamma: 构建 attention interpretability demo，帮助 evaluator 解释 emergent jump

10.3 本章小结

行动清单把讲座中的 abstract idea 变成 timeline + deliverable: 短期行动确保 governance quickly, 长期项目则把 insights 变成 publishable knowledge。

11 Emergent 能力记分板

11.1 任务追踪面板

建立 emergent capability scoreboard, 可视化多个任务的 emergent jump。以下表列出了三个常见任务及其触发条件:

任务	Trigger condition	Metrics to watch
数学推理	CoT prompt + longer tokens	Step accuracy + self-consistency
代码生成	Multi-language prompt + doc search	Syntax error rate + hallucination
对话记忆	Extended context window	Memory retention + token budget

表 15: Emergent 能力的任务追踪。

Scoreboard 的操作说明

- 每个任务用不同颜色的 line 探测 emergent jump
- 如果 multi-task jump 同时出现, 说明 architecture/data 组合成功
- 失真时立刻回到 prompt log + data cadence

11.2 指标联动与 drift analysis

Emergent score 需要同时监听 multiple metrics: accuracy/self-consistency/hallucination/token budget。Jason 展示的 drift analysis 由 4 条曲线组成, 可以快速定位 drift origin。

Drift analysis 曲线

- Accuracy curve: 观测 emergent jump
- Self-consistency: 衡量 prompt 稳定性
- Hallucination rate: 看是否 sacrifice factuality
- Token budget: 控制 compute overhead

11.3 本章小结

把 emergent events 变成 scoreboard + drift analysis, 可以把“感觉似乎好的”现象具体化: 每次 jump 都要经过 panel review, 结合 multiple metrics 判断是否值得推向 production。

12 Observation daybook 记录

12.1 关键事件记录模板

Hyung Won 分享的 daybook 模板包含: trigger description、metrics snapshot、guardrail update、next action。每次 emergent drift 或 safety incident 都需填写本模板并归档。

字段	内容
Trigger	self-consistency drop / hallucination spike
Metrics	accuracy, hallucination rate, token budget snapshot
Guardrail update	prompt change / tool constraint
Next action	retrain / rollback / new drill

表 16: Observation daybook 的模板字段。

事件记录的价值

把每个 incident 写进 daybook, 有助于 periodic review: 团队可以在 retro 会议上快速复盘 trigger → guardrail → action 的闭环, 避免重复犯错。

12.2 Lessons learned 速查

建立 ‘lessons.md’ 保存 emergent drift 的教训; Jason 推荐每次 incident 后写一段 “why/what/next”, 并按 tag 分类 (data/prompt/tool)。

Lessons learned 模板

- Why: 是什么 trigger 让 metrics 异常
- What: 我们做了什么 (prompt rollback / data filter)
- Next: 下次如何避免 (security guard / prompt stress testing)

12.3 本章小结

记录 daybook + lesson, 把 emergent/agent 的观察经过 formalization, 便于 incident review 也便于新成员快速 catch up。

13 团队承诺与治理约定

13.1 治理角色 RACI

建议把 emergent/agent 管理工作分派到 RACI: 谁负责 prompt log (Responsible)、谁支持 evaluation dashboard (Accountable)、谁提供 consult (Consulted)、谁需要 informed (Informed)。

角色	主要职责
Prompt engineer	维护 prompt versions + stress testing reports
Model engineer	保持 training log + attention monitor
Evaluator	监控 hallucination + accuracy drift
Data engineer	管理 dedup log + dataset snapshot

表 17: RACI Framework 的简版分工。

治理约定建议

- 每次 incident 后 24 小时内通知 governance channel
- Prompt log 需要 ‘version/hash + metric diff + guardrail change’
- 对所有 high-risk tool call 做 human-in-the-loop audit

13.2 本章小结

团队承诺让 governance 有人负责、有流程、有记录，而不是某个人“记在脑子里”。

14 总结与延伸

维度	核心复盘
数据/能力	Emergent 需要 multi-metric evidence + manual data review
Prompt	Instruction tuning+CoT+dashboard 组成 governance loop
架构/评估	减少归纳偏置，搭配 SLI/SLO 驱动的 evaluation pipeline

表 18: Lecture 27 三条主线回顾。

维度	风险与控制
Hallucination	taxonomy + prompt stress testing + knowledge graph check
Deployment	incident grade-book + tool stack + prompt/drift observability
Reproducibility	prompt log + incident artifact + team knowledge handoff

表 19: 风险与治理的补充视角。

14.1 拓展阅读

- Wei et al., “Emergent Abilities of Large Language Models,” 2022
- Sutton, “The Bitter Lesson,” 2019
- Chung et al., “Scaling Instruction-Finetuned Language Models” (Flan-T5), 2022
- Wei et al., “Chain-of-thought Prompting Elicits Reasoning in Large Language Models,” 2022
- Lee et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” 2022
- Brown et al., “Language Models are Few-Shot Learners,” 2020